IITM Web M.Tech Program

Reinforcement Learning Course Project

Industrial Inventory Control using Reinforcement Learning

Project objective. Develop and train an RL-based ordering policy for a warehouse handling three products. The policy must minimize inventory-related cost while remaining robust across different demand patterns, initial inventory levels and limited supplier delays.

1. Project Overview

In this project, you will apply reinforcement learning to a practical multi-product inventory-control problem. The environment follows the Gymnasium interface and represents outstanding orders using their complete arrival pipeline. Your final policy will be evaluated through a public and private leaderboard using multiple hidden scenario families.

- Formulate the inventory problem as a sequential decision-making task.
- Train an RL agent using the provided Gymnasium-compatible environment.
- Develop policies using multiple reinforcement-learning techniques that perform well across randomised and previously unseen scenarios.
- Submit separate qualifying inference policies corresponding to atleast five distinct reinforcement-learning techniques.

2. Problem Description

You control the daily replenishment decision for a warehouse that stores three products. The following conditions apply:

- Each day, you may order a quantity for each product from the permitted discrete set.
- Each product occupies a different amount of warehouse volume.
- The total inventory volume cannot exceed the warehouse capacity.
- If arrivals cause the warehouse capacity to be exceeded, excess units are discarded. The ordered units are still paid for, and a discarding cost is incurred.
- Orders arrive after a product-specific lead time. In some episodes, an order may experience a limited one-day delay.
- Daily demand varies across products and episodes.
- Costs are incurred for holding inventory, stockouts, placing orders and discarding excess stock.

3. Project Objective

Your objective is to develop an ordering policy that minimizes the total cost arising from:

- Holding excess inventory, which consumes warehouse capacity and incurs a daily holding cost.
- Stockouts, which occur when demand exceeds available inventory.
- Ordering, where a fixed cost is incurred whenever a non-zero order is placed for a product.
- Discarding, when arriving inventory causes the volume capacity to be exceeded.

4. Core Environment Configuration

The following core values remain common to all students. Episode-level randomization and assigned parameter variants are described in Sections 7, 8 and 9.

Parameter	Reference value
Number of products	3
Warehouse capacity	1000 volume units
Reference initial inventory	100 units of each product
Product volumes	Product 1: 2.0; Product 2: 3.0; Product 3: 1.5 volume units per item
Holding cost	Rs. 5.00 per unit volume per day
Stockout costs	Product 1: Rs. 400; Product 2: Rs. 500; Product 3: Rs. 300 per unfulfilled item
Fixed ordering costs	Product 1: Rs. 80; Product 2: Rs. 200; Product 3: Rs. 120 per non-zero order
Discarding costs	Product 1: Rs. 200; Product 2: Rs. 250; Product 3: Rs. 150 per discarded item
Reference lead times	Product 1: 3 days; Product 2: 2 days; Product 3: 1 day
Episode duration	50 days
Reference demand means	Product 1: 30; Product 2: 25; Product 3: 35 units per day

5. Gymnasium Environment

A Gymnasium-compatible environment will be provided. You must use the supplied environment package for training, local validation and submission-interface testing. Public and private leaderboard evaluation will use an evaluator-controlled copy of the same environment logic with hidden configurations and seeds. **DO NOT** change the core transition, observation, action logic.

```
observation, info = env.reset(seed=seed)
next_observation, reward, terminated, truncated, info = env.step(action)
```

An episode ends after 50 simulated days. When interacting with the environment, treat the episode as complete when:

```
truncated = True
```

Note on Episode Termination:
This inventory-control problem does not have a natural terminal state because warehouse operations can, in principle, continue indefinitely. Therefore, terminated will always remain False. An episode ends only when the fixed simulation horizon of 50 days is reached, in which case truncated is set to True.

6. Observation and Action Spaces

6.1 Observation

At the beginning of each decision step, the agent receives the current observation and selects the order quantities for that step. The environment then processes the day according to the sequence described in Section 10 and returns the observation for the next decision step.

The observation is a dictionary containing the information available to the agent at the beginning of the current decision step:

Observation key	Description	Shape
inventory	Current on-hand inventory for the three products.	3
arrival_pipeline	Outstanding order quantities separated by expected arrival day. The pipeline covers up to four days so that the normal maximum lead time and a possible one-day delay are represented.	3 x 4
demand_history	Demand for each product observed during the previous seven days, zero-padded at the beginning of an episode.	7 x 3
day	Current day index in the episode.	1
capacity_utilisation	Current inventory volume divided by total warehouse capacity.	1

Note on Use of Observations:
The environment provides a fixed observation containing all information available to the agent, including current inventory, the complete order-arrival pipeline, demand history, capacity utilisation and the current day. Students may choose to use all or only a subset of these observations when designing and training their policy. However, the structure of the observation returned by the environment must not be modified, and the submitted policy must rely only on the information provided through this observation.

6.2 Action

The action contains one discrete order decision for each product. Each order quantity must belong to:

```
{0, 10, 20, ..., 100}
```

Note on Order Quantities:
Internally, the Gymnasium environment represents each product action as an integer from 0 to 10. The environment maps action index a_i to an order quantity of $10a_i$ units. For example, the environment action [4, 2, 0] represents order quantities [40, 20, 0].

For leaderboard submission, however, run_policy() must return the actual order quantities [q1, q2, q3], with each quantity belonging to {0, 10, 20, ..., 100}. The evaluator will perform the required conversion to the internal environment action representation.

7. Episode-Level Domain Randomization

The environment will not use one fixed demand sequence or one fixed starting state during training. At the beginning of each episode, selected parameters will be randomized within disclosed limits. This is intended to reward robust policies rather than policies tuned to a single sequence.

Randomized element	Rule
Demand level	A product-specific multiplier between 0.85 and 1.15 is applied to the reference demand mean.
Initial inventory	The starting inventory of each product is sampled in multiples of 10 from 80 to 120 units.
Lead-time uncertainty	An order may be delayed by exactly one additional day. The episode-specific delay probability lies between 0% and 10%.
Random seed	Different seeds generate different demand realizations and delay events.

Important: The randomization ranges are public, but the exact episode parameters and seeds used for leaderboard evaluation will not be disclosed.

8. Demand Scenario Families

Training and evaluation episodes may be generated using one or more of the following scenario characteristics. Students are expected to test their policies across multiple declared scenario families using the facilities provided in the project package. The exact scenario characteristics used in each episode, their parameter values, sequence and random seeds for the public and private leaderboards will remain hidden.

Scenario family	Description
Stationary demand	Daily demand fluctuates around a stable mean for the full episode.
Seasonal demand	Demand follows a repeating weekly pattern around the episode-specific mean.
Gradual trend	Demand gradually increases or decreases over the episode.
Temporary demand shock	One product experiences a temporary increase in demand for a limited number of days.

For example, an episode could have:

- seasonal demand only,
- increased demand with a temporary shock,

- seasonal demand combined with a gradual upward trend.

Note: The private leaderboard will not introduce a new inventory rule that is absent from the project specification. It will use hidden instances and mixtures of the declared scenario families and randomization ranges.

9. Assigned Parameter Variant

Each student will be assigned one parameter variant through a configuration file supplied with the project package. The variant will be determined automatically from the student registration number. The purpose of the assigned variant is to discourage direct copying of trained policies, tuned hyperparameters or hard-coded solutions while ensuring that all students solve the same fundamental reinforcement-learning problem.

- The assigned variant changes only the demand multipliers, initial-inventory profile and lead-time delay profile within the ranges declared above.
- The core state representation, action space, transition rules, ordering constraints, cost components and reward definition remain identical for all students.
- The assigned variant must be used consistently in the student's notebook, training code, local experiments, validation results and submitted technical report.
- The submitted policy must not contain manually altered environment parameters or be configured using the variant assigned to another student.
- Public and private leaderboard evaluation will be carried out using common evaluator-controlled parameter configurations that are different from, and not materially close to, any student-assigned variant.
- The same public leaderboard configurations will be used for all students, and the same private leaderboard configurations will be used for all students.
- A submission may be rejected or excluded from evaluation if it:
    - uses another student's assigned variant;
    - alters the assigned configuration in the submitted training evidence;
    - hard-codes actions for particular public or private evaluation cases;
    - does not conform to the prescribed policy interface;
    - misrepresents the reinforcement-learning technique used;
    - duplicates the same underlying technique across multiple qualifying submissions without substantial methodological distinction.

10. Environment Sequence within Each Day

After the agent selects an action based on the current observation, each call to step(action) performs the following operations:

1. Receive orders whose scheduled arrival day has been reached.
2. Enforce warehouse capacity after the day's arrivals and calculate discarded quantities.
3. Receive the new order action and add the ordered quantities to the future arrival pipeline.
4. Generate the day's demand according to the selected scenario family.
5. Serve demand using available inventory and record any unfulfilled demand.
6. Update inventory, demand history, pipeline positions and capacity utilisation.
7. Calculate the cost, reward and information returned by the environment.

11. Reward Function

The official reward is the negative of the total daily cost, scaled by 100:

```
daily_cost = holding_cost + stockout_cost + ordering_cost + discard_cost
base_reward = -daily_cost / 100
episode_return = sum(base_reward over 50 days)
```

The cost components are calculated as follows:

- Holding cost: remaining inventory volume multiplied by the daily holding cost.
- Stockout cost: unfulfilled quantity multiplied by the product-specific stockout cost.
- Ordering cost: product-specific fixed cost applied when the corresponding order quantity is greater than zero.
- Discarding cost: discarded quantity multiplied by the product-specific discarding cost.

Note: Although the agent may be trained by maximising episode return, leaderboard ranking will use the corresponding unscaled total episode cost. Since the official reward is the negative scaled cost, maximising official return and minimising official cost are equivalent.

Training reward shaping. You may use a surrogate or shaped reward during training. However, all leaderboard evaluation will use only the official base reward and cost function shown above.

12. Leaderboard Evaluation

Each submitted policy will be evaluated using a private copy of the inventory-control environment. The evaluator will execute deterministic episodes generated in advance using hidden scenario configurations and random seeds.

Evaluation sets

- The public leaderboard will contain **20 deterministic evaluation episodes**.
- The private leaderboard will contain a separate set of **20 deterministic evaluation episodes**.
- The public and private episode sets will be mutually exclusive.
- The same public episodes will be used to evaluate every eligible submission.
- The same private episodes will be used to evaluate every eligible submission.
- Public and private evaluation will contain multiple declared scenario families and will not consist merely of different random seeds from one fixed demand process.
- The public and private scenario configurations will be different from, and not materially close to, the parameter variants assigned to individual students.
- Episode-level demand sequences, scenario parameters, seeds, initial conditions and detailed cost breakdowns will not be released.

Each evaluation episode will use a fixed scenario configuration, initial state, demand sequence and realised delay sequence. Submitted inference policies must use deterministic action selection. Therefore, re-evaluating the same qualifying policy on the same episode will produce the same cost.

Note: A technique may use stochastic exploration during training, but the submitted run_policy() function must perform deterministic inference.

Eligible technique submissions

Each student may submit policies developed using multiple reinforcement-learning techniques. The list of techniques that are considered distinct are given in the next section. For leaderboard scoring, the system will automatically identify the student's **best five qualifying submissions**, subject to the following conditions:

- each selected submission must represent a distinct reinforcement-learning technique;
- each submission must comply with the prescribed policy interface;
- where a student makes multiple submissions using the same technique, only the best-performing eligible submission from that technique may be considered;
- the technique associated with every submission must be declared correctly;
- the evaluator may verify the submitted code and report to confirm that the declared techniques are genuinely distinct.

Public leaderboard score

- Each of the student's five selected public-leaderboard submissions will be evaluated over all **20 public episodes**.
- Students will be ranked on the public leaderboard according to this overall average cost. **Lower average cost indicates better performance.**

Private leaderboard score

- After the final submission deadline, the same five qualifying technique submissions will be evaluated over all **20 private episodes**.
- Students will be ranked on the private leaderboard according to this overall average cost. **Lower average cost indicates better performance.**
- The private leaderboard results will be revealed only after the final submission deadline.

Note: If a student has fewer than five eligible distinct-technique submissions, the leaderboard average will be calculated using only the available qualifying submissions. However, **two marks** will be deducted from the student's total project marks for each missing distinct-technique submission.

Unique Techniques That May Be Attempted

Students are encouraged to explore multiple distinct solution techniques during the project. A technique will be considered unique only when it differs meaningfully in its decision-making or learning approach. Changing only hyperparameters, random seeds, number of training episodes, neural-network size or reward-shaping coefficients will not normally be treated as a separate technique. The following will be considered distinct techniques that one can use in this project:

1. Tabular Q-Learning
2. Tabular SARSA
3. TD(λ) with Eligibility Traces
4. Neural Network based Q-Learning
5. Neural Network based SARSA
6. REINFORCE with or without a baseline
7. A2C
8. A3C
9. Proximal Policy Optimizaton (PPO)
10. DQN

11. Double DQN

The following will generally not be considered distinct techniques:

- The same algorithm with different learning rates
- The same algorithm with different neural-network sizes
- The same method trained for different numbers of episodes
- The same policy with different random seeds
- The same code with renamed functions or reorganized statements

For each technique submitted, students should clearly identify:

1. The technique used.
2. The observation variables used by the policy.
3. The reward used during training.
4. The main parameters or hyperparameters.
5. The resulting validation or leaderboard performance.

13. Project-mark distribution

The total project marks will be distributed as follows:

Project component	**Weight**
Submitted code and project report	20%
Public leaderboard performance based on the best five distinct technique submissions	40%
Private leaderboard performance based on the same five distinct technique submissions	40%
Total	**100%**

The conversion of leaderboard performance and position to marks will be decided at the end of the project depending on the overall performance of the class. The code-and-report component will assess aspects such as:

- correctness and completeness of the implementation;
- successful execution of the submitted policies;
- appropriate use of five distinct reinforcement-learning techniques;
- experimental methodology;
- comparison of techniques;
- analysis of results;
- code quality and reproducibility;
- interpretation of policy behaviour;
- discussion of limitations.

14. Policy Submission Interface

Every leaderboard submission must contain one policy file implementing the prescribed interface. Each submission must also declare its technique category and submission identifier through the leaderboard portal or the required metadata file.

The policy file must contain the following function:

```
def run_policy(observation):
    """Return order quantities for Products 1, 2 and 3."""
    return [q1, q2, q3]
```

- Each returned quantity must be one of 0, 10, 20, ..., 100.
- The evaluator will convert these quantities to the internal MultiDiscrete action indices.
- The function will receive only the documented observation; it will not receive future demand, scenario identity or hidden evaluation parameters.
- The function must not call env.step() or env.reset().
- The evaluator will start a fresh policy process at the beginning of every episode.
- Training must not occur inside run_policy(). The function should perform inference only.
- Model parameters may be loaded once when the policy module is imported. They must not be retrained or modified during evaluation.
- Internet access and access to files outside the submitted package will not be available during evaluation.

15. Submission Requirements

15.1 One Jupyter Notebook (.ipynb)

The notebook must include:

- reproducible training pipelines for the five submitted techniques;
- assigned parameter variant and configuration-loading method;
- common state preprocessing and action mapping;
- technique-specific model architecture or policy representation;
- important hyperparameters;
- reward shaping, where used;
- training and local-validation results;
- learning curves or convergence evidence where relevant;
- comparison table covering all five techniques;
- code or export steps used to generate each final submitted policy;
- clear mapping between each technique, leaderboard submission identifier and model artefact.

15.2 Policy File (.py)

The final project package must contain the policy files and model artefacts corresponding to the five frozen distinct-technique submissions used for public and private evaluation.

15.3 Project Report (.pdf or .docx)

The project report must be concise and must not exceed **four pages**, excluding an optional title page and references. A report of approximately **three to four pages** is expected.

The report should focus on:

- what the student implemented;
- how the experiments were conducted;

- what results were obtained;
- what the student inferred from those results.

The inventory-control problem and each reinforcement-learning technique should be introduced in no more than one or two sentences. Students are not required to provide detailed descriptions of the problem, standard reinforcement-learning concepts or textbook explanations of the techniques.

The report should contain:

1. **Implementation summary**
    - Assigned parameter variant
    - Names of the five distinct reinforcement-learning techniques
    - One or two lines describing how each technique was implemented or adapted
2. **Experimental approach**
    - Important preprocessing or state-representation choices
    - Reward shaping, where used
    - Main hyperparameters or training choices that materially affected performance
    - Local validation approach
3. **Results**
    - A compact comparison table for the five techniques selected on the public leaderboard
    - Relevant learning curves or other essential plots
    - Any important validation results
4. **Observations and inference**
    - Which techniques performed better or worse
    - Likely reasons for the observed differences
    - Important policy behaviours noticed
    - Key lessons from the project

Note: Source code, complete hyperparameter listings and routine execution output should remain in the notebook and should not be reproduced in the report.

16. Important Rules

- Use only the official environment package and your assigned configuration for reported experiments.
- Do not modify the official cost function or leaderboard evaluator.
- Each of the five qualifying submissions must be reproducible from the submitted project package and must correspond to the technique declared on the leaderboard portal.
- Do not share policy code, trained models or notebooks with other students.
- Submissions may be checked automatically for source-code, notebook and policy-behaviour similarity after removing instructor-provided boilerplate.
- All submitted work must comply with the academic-integrity and permitted-tool-use rules announced for the course.

17. Files Provided to Students

File or folder	Purpose
industrial_inventory_env	Gymnasium-compatible environment package.
starter_notebook.ipynb	Provides environment-loading and basic interaction examples. It also contains the approved configuration-generation utility through which students enter their official roll number and generate their assigned parameter variant.
policy_validation_tests.py	Checks whether the submitted policy file loads correctly, implements the required run_policy(observation) interface, returns valid actions, performs deterministic inference and satisfies basic runtime requirements. Students are expected to implement their own local evaluation loop for comparing policies across training and validation episodes.
submission_template.py	Required run_policy function template.
requirements.txt	Permitted package versions.

The detailed schedule, submission deadlines, file-size limits and leaderboard submission cap will be announced separately on the course portal.